

Asistente de Ayuda IA — Neura / Zentra ERP

Resumen completo de la sesión de implementación y activación del MVP (Fase 1)

Fecha: 2026-06-09 · Proyecto: C:\Neura\neura-erp-sistemas-propio · Schema de datos: neura · Rama: main

1. Punto de partida

Se retomó el proyecto del **asistente de ayuda con Claude** embebido en el ERP. El MVP (Fase 1) ya estaba **construido en código local pero sin desplegar, sin commitear y sin base de datos aplicada**. El objetivo de la sesión fue **activarlo y verificarlo end-to-end en local**.

Piezas que ya existían en el working tree:

Pieza	Archivo
Migración BD (tablas + RPC de búsqueda)	supabase/migrations/20260605120000_assistant_module.sql
Script de ingesta del corpus	scripts/assistant-ingest.ts
Wrapper para aplicar la migración por schema	scripts/apply-assistant-module.cjs
Smoke test (valida Claude sin BD)	scripts/assistant-smoke.ts
Endpoint de chat (SSE streaming)	src/app/api/assistant/chat/route.ts
Widget flotante	src/components/assistant/AssistantWidget.tsx
Montaje en AppShell (gated por env)	src/components/AppShell.tsx

2. Lo que hicimos, paso a paso

Paso 0 — Smoke test de la integración con Claude ✓ OK

Ejecutamos `npx tsx scripts/assistant-smoke.ts`. Claude (`claude-haiku-4-5`) respondió correctamente usando el corpus real (`facturas.md`), con pasos correctos y cita de fuente. Costo: 1739 tokens in / 338 out. Confirmó que la `ANTHROPIC_API_KEY` y el modelo funcionan.

Paso 1 — Generación del SQL (opción manual, sin conectar a la BD)

Como no había `SUPABASE_DB_URL` en `.env.local`, se eligió generar el SQL y ejecutarlo manualmente en el SQL editor de Supabase. Generamos apuntando al schema `neura`:

```
node scripts/apply-assistant-module.cjs neura --print > assistant-migration.neura.sql
npx tsx scripts/assistant-ingest.ts --schema=neura --emit-sql=assistant-corpus.neura.sql
```

El wrapper reescribe el template (`zentra_erp` → `neura`) automáticamente. Resultado: **0 ocurrencias de `zentra_erp`** en los archivos generados.

Paso 2 — Aplicación de la migración y carga del corpus ✓ OK

Orden de ejecución en el SQL editor: **1°** `assistant-migration.neura.sql` (crea las 4 tablas + índices + RLS + RPC), **2°** `assistant-corpus.neura.sql`. Verificación confirmada: **15 documentos / 116 chunks**.

Paso 3 — Fix de permisos (error 42501) ✓ Resuelto

Problema detectado: al probar la conectividad vía PostgREST con el `service_role`, las tablas devolvían 42501: permission denied for table assistant_kb_documents .

Causa: el schema `neura` no tiene *default privileges* que otorguen permisos automáticos a los roles de PostgREST, así que las tablas nuevas (creadas por el rol `postgres` desde el SQL editor) quedaron sin `GRANT` .

Solución: se agregó una sección de `GRANT` idempotente a la migración (template) y se ejecutó `assistant-grants.neura.sql`, que otorga: a `service_role` acceso completo a las 4 tablas + `execute` sobre la RPC; a `authenticated` solo `select` sobre las 2 tablas del corpus. **No** toca otros schemas ni otras tablas.

Paso 4 — Fix de recall en la búsqueda ✓ Resuelto

Problema detectado: la RPC `assistant_search_kb` devolvía `[]` con preguntas naturales como "como hago una nota de credito".

Causa: `websearch_to_tsquery` une los términos con **AND**; al exigir que TODAS las palabras estén en el mismo chunk, un verbo ausente del corpus ("hago") vaciaba el resultado.

Solución: se reescribió la función para matchear con **OR** (recall) pero **rankear más alto los chunks que satisfacen el AND completo** (precisión). Técnica: tomar la tsquery AND, reemplazar `&` por `|` y castear de vuelta a `tsquery`. Se aplicó con `assistant-fn-update.neura.sql`.

Verificación posterior (preguntas naturales): "nota de credito" → facturas ✓, "alta de producto" → inventario ✓, "agregar cliente" → ventas/clientes ✓.

Paso 5 — Configuración de entorno local y arranque ✓ OK

Se completó `.env.local` con las 7 variables necesarias y se levantó el dev server: **Next.js 16.1.6** listo en `http://localhost:3000`.

3. Estado actual — verificado

Capa	Estado	Cómo se verificó
Tablas <code>assistant_*</code> en <code>neura</code>	✓ creadas	HTTP 200 vía PostgREST
Grants <code>service_role</code> / <code>authenticated</code>	✓ aplicados	resuelto el 42501
Corpus cargado	✓ 15 docs / 116 chunks	count en SQL editor
RPC <code>assistant_search_kb</code> (con fix de recall)	✓ relevante	3 preguntas naturales devuelven chunks correctos
Integración con Claude	✓ responde	smoke test (1739/338 tokens)
URL base de la API Supabase	✓ validada	<code>https://api.neura.com.py</code>
Dev server local	✓ corriendo	<code>http://localhost:3000</code>
Prueba en navegador (widget vivo)	🕒 pendiente	requiere login del usuario en el browser

4. Archivos creados / modificados en esta sesión

Modificados (template de migración)

- `supabase/migrations/20260605120000_assistant_module.sql` — se le agregaron: **(a)** sección de `GRANT` idempotente para roles PostgREST; **(b)** reescritura de la RPC `assistant_search_kb` con OR-recall + boost por AND completo.
- `.env.local` (gitignored — NO se commitea) — flags del asistente + credenciales Supabase.

Generados (artefactos de trabajo, en la raíz del repo)

- `assistant-migration.neura.sql` — migración completa para `neura` (tablas + grants + RPC mejorada).
- `assistant-corpus.neura.sql` — corpus (15 docs / 116 chunks), refresh transaccional.
- `assistant-grants.neura.sql` — solo los GRANT (fix del 42501).
- `assistant-fn-update.neura.sql` — solo la RPC (fix de recall).

5. Variables en `.env.local` (local, gitignored)

Variable	Para qué
<code>APP_DB_SCHEMA=neura</code>	schema de datos del ERP en esta instancia
<code>ANTHROPIC_API_KEY</code>	API de Claude (modelo del asistente)
<code>ASSISTANT_ENABLED=1</code>	habilita el endpoint <code>/api/assistant/chat</code>
<code>NEXT_PUBLIC_ASSISTANT_ENABLED=1</code>	muestra el widget flotante
<code>NEXT_PUBLIC_SUPABASE_URL=https://api.neura.com.py</code>	URL base de la API (gateway PostgREST/Auth)
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	key pública (auth del cliente)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	key privada (endpoint: retrieval + persistencia)

Seguridad: la `service_role` es de máximo privilegio. `.env.local` está gitignored y solo vive en la máquina local. En el hosting (producción) estas variables van en el panel del proveedor, nunca en el repo.

6. Lo que falta para terminar

1. **AHORA Probar el widget en el navegador:** abrir `http://localhost:3000`, loguearse, y verificar que el botón flotante aparece abajo a la derecha y responde con cita de fuente.
2. **Decidir despliegue a producción** (requiere autorización expresa del propietario — regla del proyecto). Implica:
 - Cargar en el panel del hosting (Vercel): `ANTHROPIC_API_KEY`, `ASSISTANT_ENABLED=1`, `NEXT_PUBLIC_ASSISTANT_ENABLED=1` (las de Supabase ya deberían existir allí).
 - **Commitear el código del MVP** (endpoint, widget, scripts, migración) — pendiente de OK.
 - Deploy y prueba con 1–2 empresas piloto.
3. **Limpieza:** los 4 archivos `assistant-*.neura.sql` de la raíz son artefactos temporales; decidir si se borran o se archivan (no deberían commitearse a la raíz).

7. Notas de diseño relevantes

- **Schema dinámico:** la migración usa `zentra_erp` como template; los wrappers (`apply-assistant-module.cjs`, `assistant-ingest.ts`) lo reescriben al schema real (`neura`). En runtime, `createServiceRoleClient` usa `db.schema = APP_DB_SCHEMA`, así que las llamadas del endpoint van a `neura` vía PostgREST sin tocar configuración.
- **Doble flag de seguridad:** con los flags apagados el código es inerte (widget no se monta, endpoint responde 404).

- **Aislamiento multi-tenant:** el corpus es global y sin datos de clientes; los módulos habilitados del tenant solo *filtran* qué documentación se recupera. Conversaciones se guardan con `empresa_id` en tablas deny-by-default (solo service role).
- **Cuota:** `ASSISTANT_DAILY_LIMIT` (default 50 mensajes/usuario/día).
- **Retrieval léxico** (tsvector español), sin embeddings en esta fase. Queda como fase futura si el recall léxico se queda corto.

8. Cómo continuar en la nueva cuenta de Claude

Contexto mínimo para retomar:

- El MVP está **funcionando end-to-end en local**; falta solo la prueba visual en el navegador y la decisión de despliegue.
- Toda la lógica de BD (tablas, grants, corpus, RPC mejorada) **ya está aplicada en la instancia** `neura` de Supabase.
- Regla del proyecto: **NADA de commit / push / deploy / migraciones sin autorización expresa del propietario.**
- El dev server local se levanta con `npm run dev` (lee `.env.local`).
- Para regenerar el corpus tras editar los `.md`: `npx tsx scripts/assistant-ingest.ts --schema=neura` (requiere `SUPABASE_DB_URL`) o `--emit-sql` para el editor.

Documento de traspaso generado automáticamente · Sesión del 2026-06-09 · El detalle funcional de cada módulo y el diseño de arquitectura están en `docs/assistant/` (architecture.md, recommendations.md, system-map.md, IMPLEMENTATION.md y los 13 docs de módulos).